

文档三：详细设计报告

1. 系统方案设计

1.1 项目背景

OfferBot 是一个基于微信公众号的 Offer 信息交互系统，用户只需要在微信公众号会话中输入中文命令，即可完成 Offer 信息的查询、上传、详情查看、更新、替换、删除和批量上传。系统将微信公众号会话抽象为一个轻量级命令行入口，后端负责命令解析、业务校验、权限控制、数据库读写和回复文本生成。

本系统重点实现以下能力：

- 使用微信公众号作为交互前端。
- 使用中文命令行格式完成核心业务操作。
- 使用 MySQL 持久化 Offer、微信用户和命令日志。
- 通过 `openid` 记录上传者，限制普通用户只能修改或删除自己上传的 Offer。
- 支持分页、部分检索、批量上传、错误提示和请求频率控制。
- 通过 Docker Compose 部署 Spring Boot 服务和 MySQL 数据库。

1.2 技术路线

1.2.1 前端交互：微信公众号

系统前端采用微信公众号聊天窗口。用户通过文字消息输入命令，微信服务器将消息转发到后端配置的 URL，后端处理后返回 XML 文本回复。

选择微信公众号作为交互端的原因：

- 部署和使用成本低，用户只需关注公众号。
- 聊天场景天然适合命令式交互。
- 微信官方提供消息推送、签名校验和用户 `openid`，便于完成身份区分。

1.2.2 后端：Java + Spring Boot

后端使用 Java 17 和 Spring Boot 3.3.5。主要原因：

- Spring Boot 适合快速开发 Web 后端，工程结构清晰。
- 通过 Controller、Service、Mapper 分层实现业务，便于维护和测试。

- Java 类型系统较强，适合将命令解析结果、Offer 草稿、分页结果等结构化表达。
- 后续如果扩展管理后台、定时任务或接入更多接口，Spring 生态支持较完整。

1.2.3 数据库：MySQL + Flyway

数据库采用 MySQL 8.4，使用 Flyway 管理建表脚本。数据库中主要保存三类数据：

- `wx_user`：微信公众号用户状态、角色和请求频率窗口。
- `offer`：Offer 信息和上传者 `source_openid`。
- `command_log`：每一次命令执行记录，便于审计和排查问题。

MySQL 相比本地文件型数据库更适合服务器部署，也方便使用 Navicat 等工具查看数据。Flyway 用于保证数据库结构可重复创建和迁移。

1.2.4 数据访问：MyBatis-Plus

系统使用 MyBatis-Plus 操作数据库。实体类通过 `@TableName` 映射表名，Mapper 继承 MyBatis-Plus 的基础能力。Offer 查询使用 `LambdaQueryWrapper` 拼接条件，避免手写 SQL 字符串。

1.2.5 部署：Docker Compose

系统部署由 Docker Compose 编排两个容器：

- `offerbot`：Spring Boot 后端服务，监听容器内 8081 端口。
- `offerbot-mysql`：MySQL 8.4 数据库，数据卷持久化。

服务器上通过 Nginx 和 HTTPS 域名将 <https://offerbot.kyy008.me/wechat/> 转发到本地后端端口。微信公众号后台配置该 URL 作为消息推送地址。

1.3 系统外部架构

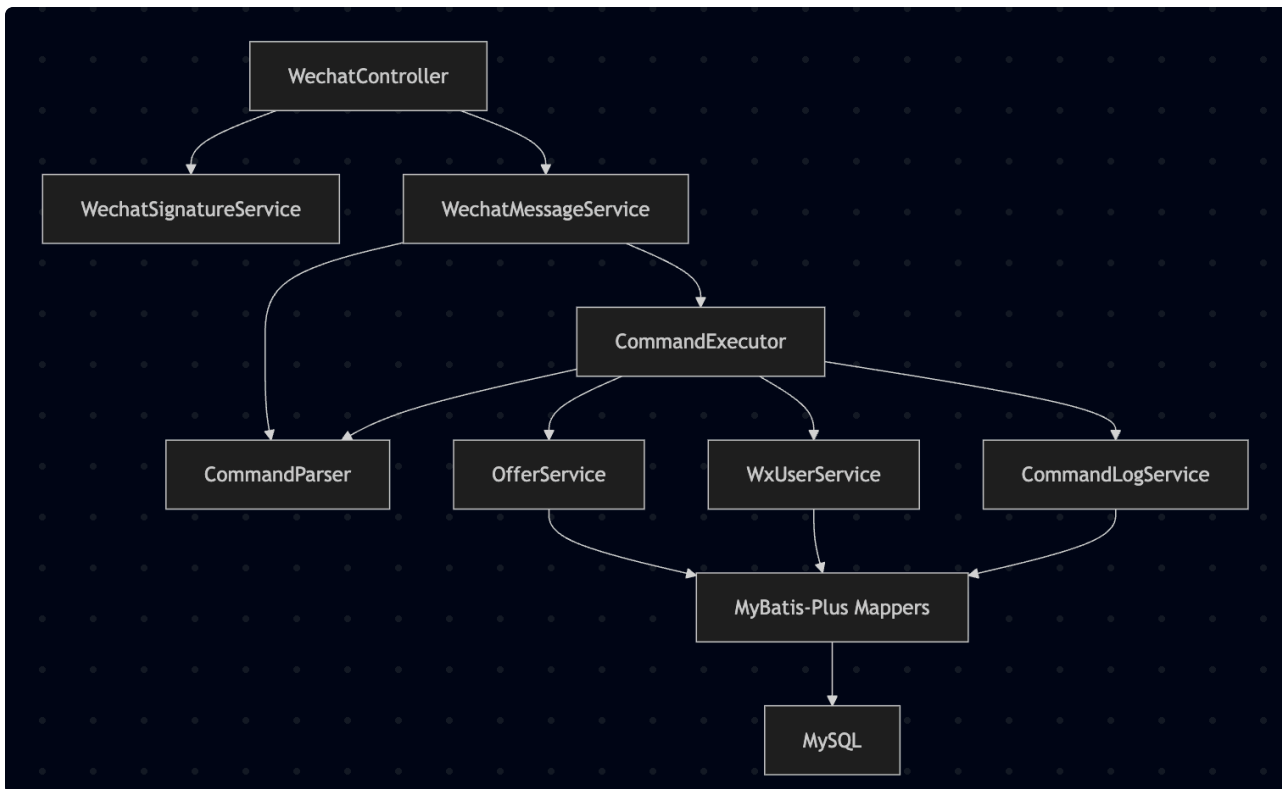


外部系统中的主要职责如下：

- 用户：发送中文文本命令。
- 微信公众号：提供聊天入口。

- 微信服务器：完成消息中转，并附带签名、时间戳、随机串和用户 `openid`。
- Nginx：提供 HTTPS 入口并反向代理到后端。
- OfferBot 后端：校验签名、解析 XML、执行命令、访问数据库、生成回复。
- MySQL：保存 Offer、用户状态和命令日志。

1.4 系统内部架构



系统采用典型分层结构：

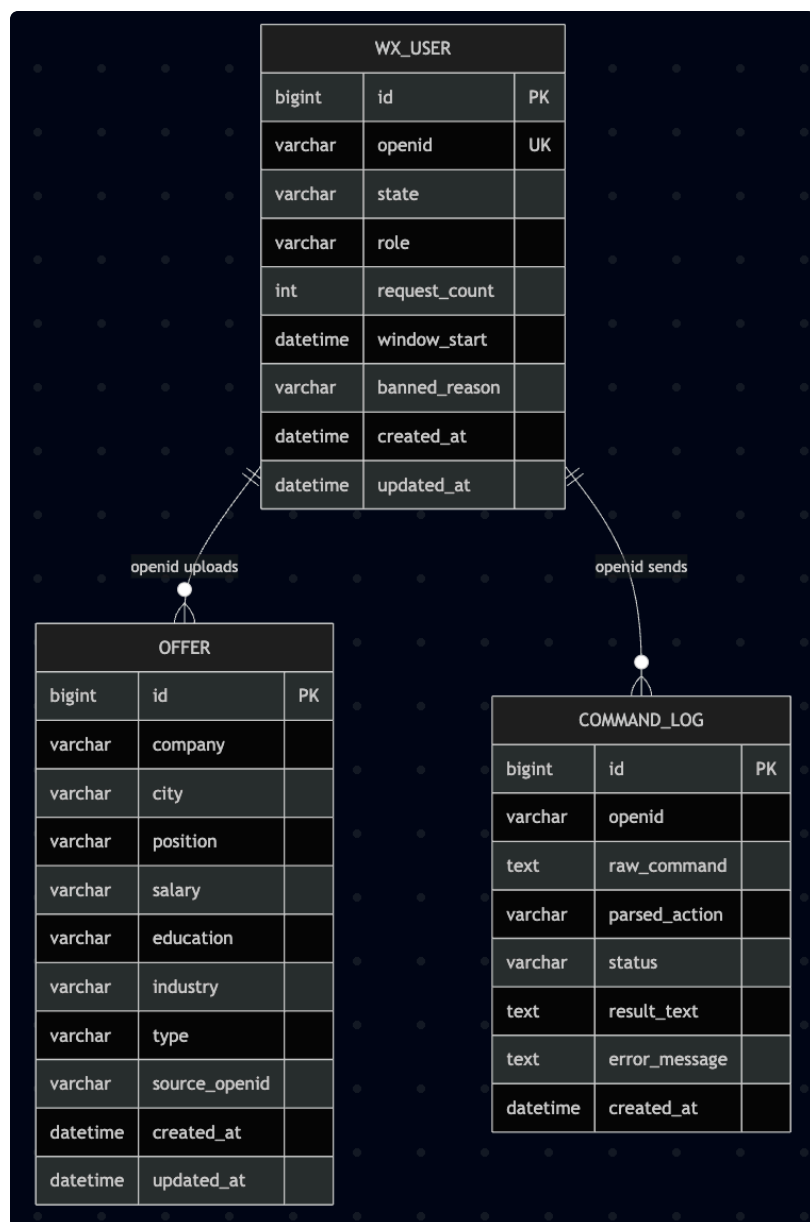
- Controller 层：处理 HTTP 请求和微信服务器校验。
- Message 层：解析微信 XML 消息，并生成微信 XML 回复。
- Command 层：将用户文本解析为结构化命令，并分发到业务服务。
- Service 层：实现 Offer、用户、日志的核心业务。
- Mapper 层：负责数据库访问。
- Domain/DTO 层：表示实体对象和业务传输对象。

1.5 代码模块划分

模块	主要文件	职责
启动模块	OfferBotApplication	Spring Boot 应用入口
控制器模块	WechatController , HealthController	微信接口、健康检查
微信服务模块	WechatSignatureService , WechatMessageService	签名校验、XML 解析、XML 回复
命令模块	CommandParser , CommandExecutor , ParsedCommand , CommandAction	文本命令解析、命令执行分发、回复格式化
Offer 业务模块	OfferService	Offer 增删改查、分页、检索、批量上传、校验
用户业务模块	WxUserService	用户自动创建、状态记录、频率检测
日志模块	CommandLogService	记录命令原文、解析动作、成功或失败原因
数据模型模块	Offer , WxUser , CommandLog	数据库实体映射
数据访问模块	OfferMapper , WxUserMapper , CommandLogMapper	MyBatis-Plus 数据访问

2.数据库详细设计

2.1 E-R 关系



说明：当前数据库设计没有强制外键约束，而是通过 `openid` 进行逻辑关联。这样可以降低测试、数据清理和演示数据导入的复杂度。

2.2 `wx_user` 表

`wx_user` 用于记录微信公众号用户状态。

字段	类型	说明
id	BIGINT	主键，自增
openid	VARCHAR(128)	微信用户唯一标识，唯一索引
state	VARCHAR(32)	用户状态：用来标志用户是否有爬虫 / 违规行为
role	VARCHAR(32)	用户角色：USER、ADMIN
request_count	INT	当前频率窗口内请求次数
window_start	DATETIME	当前请求频率窗口开始时间
banned_reason	VARCHAR(255)	被限制原因
created_at	DATETIME	创建时间
updated_at	DATETIME	更新时间

用户第一次发送消息时会自动创建记录。普通用户默认 state=NORMAL 、 role=USER 。

2.3 offer 表

offer 保存系统核心业务数据。

字段	类型	说明
id	BIGINT	Offer 编号，主键，自增
company	VARCHAR(128)	公司名称，必填
city	VARCHAR(64)	城市，必填
position	VARCHAR(128)	岗位，必填
salary	VARCHAR(128)	薪资，必填，新增或修改时要求符合 数字/天、数字/月、数字/年
education	VARCHAR(64)	学历要求，可为空
industry	VARCHAR(64)	公司类型，可为空
type	VARCHAR(32)	岗位类型，可为空，系统会将“实习、校招、社招”规范化为内部值
source_openid	VARCHAR(128)	上传者 openid，用于权限控制
created_at	DATETIME	创建时间
updated_at	DATETIME	更新时间

索引设计：

- idx_offer_source_openid：支持按上传者查询和权限判断。
- idx_offer_company：支持公司维度检索。
- idx_offer_city_position：支持城市和岗位组合检索。

2.4 command_log 表

command_log 记录每次命令执行过程。

字段	类型	说明
id	BIGINT	主键，自增
openid	VARCHAR(128)	发起命令的微信用户
raw_command	TEXT	用户输入原文
parsed_action	VARCHAR(64)	解析后的动作，例如 QUERY 、 CREATE
status	VARCHAR(32)	执行状态： SUCCESS 或 FAILED
result_text	TEXT	成功回复文本
error_message	TEXT	失败原因
created_at	DATETIME	日志创建时间

日志表建立 idx_command_log_openid_created_at 索引，便于按用户和时间排查问题。

3.接口详细设计

3.1 对外 HTTP 接口

3.1.1 微信服务器验证接口

项目	内容
URL	/wechat 、 /wechat/
方法	GET
参数	signature 、 timestamp 、 nonce 、 echostr
成功返回	原样返回 echostr
失败返回	HTTP 403，内容为 invalid signature

处理流程：

1. 微信服务器向配置 URL 发起 GET 请求。
2. 后端读取 `signature`、`timestamp`、`nonce`。
3. `WechatSignatureService` 将 Token、timestamp、nonce 字典序排序后拼接。
4. 对拼接结果做 SHA-1。
5. 与 `signature` 对比，相同则验证通过并返回 `echostr`。

3.1.2 微信消息接收接口

项目	内容
URL	<code>/wechat</code> 、 <code>/wechat/</code>
方法	POST
请求体	微信 XML 消息
返回类型	XML 文本消息
签名失败	HTTP 403

处理流程：

1. Controller 先进行签名校验。
2. `WechatMessageService` 使用 DOM 解析 XML，读取 `ToUserName`、`FromUserName`、`MsgType`、`Content`、`Event`。
3. 如果是关注事件，返回关注提示。
4. 如果不是文字消息，提示当前仅支持文字命令。
5. 如果是文字消息，将 `FromUserName` 作为 `openid`，将 `Content` 作为命令文本交给 `CommandExecutor`。
6. 生成微信 XML 文本回复。

3.1.3 健康检查接口

项目	内容
URL	/health
方法	GET
成功返回	OK

该接口用于部署后确认服务是否启动。

3.2 对内业务 API

3.2.1 OfferService

方法	功能
createOffer(OfferDraft draft, String sourceOpenid)	创建 Offer，并记录上传者
getOffer(Long id)	按编号查询单条 Offer
listOffers(OfferSearchCriteria criteria, Integer page, Integer size)	按条件分页查询 Offer
updateOffer(Long id, Map<String, String> patch, String actorOpenid, boolean admin)	局部更新 Offer
replaceOffer(Long id, OfferDraft draft, String actorOpenid, boolean admin)	整体替换 Offer
deleteOffer(Long id, String actorOpenid, boolean admin)	删除 Offer
batchCreateOffers(List<String> rows, String sourceOpenid)	批量创建 Offer

核心设计点：

- 创建、替换时需要校验必填字段。
- 局部更新时只更新用户提供的字段。
- 删除、更新、替换都要先检查当前用户是否是上传者；管理员可以越权操作。
- 查询默认按 id 升序返回。

- 分页默认 `page=1`、`size=5`，最大 `size=10`。

3.2.2 WxUserService

方法	功能
<code>recordRequest(String openid)</code>	自动创建或更新用户请求窗口
<code>findByOpenid(String openid)</code>	按 openid 查询用户
<code>updateState(String openid, String state)</code>	修改用户状态

频率控制规则：

- 每个用户维护一个 30 秒请求窗口。
- 如果 30 秒内请求超过 30 次，且用户当前为 `NORMAL`，则设置为 `SPIDER`。
- `SPIDER` 用户不能查询 Offer。
- `BANNED` 用户不能上传、更新、替换、删除或批量上传。

3.2.3 CommandLogService

方法	功能
<code>log(...)</code>	记录命令原文、动作、状态、成功结果或失败原因

`CommandExecutor` 使用 `finally` 写日志，因此业务成功和失败都会被记录。

4.命令解析与交互设计

4.1 命令格式

系统采用中文命令，参数使用 `字段=值` 形式。示例：

Plain Text

- 1 查Offer 关键词=字节 岗位类型=实习 页码=1 每页=5
- 2 上传 公司=字节跳动 城市=北京 岗位=后端开发实习生 薪资=300/天 学历要求=本科 公司类型=互联网 岗位类型=实习
- 3 详情 编号=1
- 4 更新 编号=1 薪资=350/天
- 5 删除 编号=1

命令之间使用空格分隔，系统会将全角空格替换为普通空格。参数顺序不强制，用户可以按自己习惯输入。

4.2 字段映射

CommandParser 会把中文字段映射为内部字段。

中文字段	内部字段	说明
编号	id	Offer 编号
公司	company	公司名称
城市	city	城市
岗位	position	岗位
薪资	salary	薪资
学历要求、 学历	education	学历要求
公司类型、 行业	industry	公司类型
岗位类型、 类型	type	岗位类型
关键词	keyword	模糊检索关键词
页码	page	当前页
每页	size	每页数量

系统主推字段名称为 学历要求、 公司类型、 岗位类型，同时保留少量旧别名，避免历史命令完全失效。

4.3 支持的命令动作

用户命令	内部动作	说明
<code>帮助</code> 、 <code>帮助 1</code>	<code>HELP</code>	查看总帮助或指定功能帮助
<code>查Offer</code>	<code>QUERY</code>	条件查询 Offer
<code>列表</code>	<code>LIST</code>	查看 Offer 列表
<code>上传</code>	<code>CREATE</code>	新增一条 Offer
<code>详情</code>	<code>GET</code>	查看单条 Offer
<code>更新</code>	<code>UPDATE</code>	局部更新一条 Offer
<code>替换</code>	<code>REPLACE</code>	整体替换一条 Offer
<code>删除</code>	<code>DELETE</code>	删除一条 Offer
<code>批量上传</code>	<code>BATCH_CREATE</code>	多行批量新增 Offer

4.4 查询设计

`查Offer` 支持以下主要字段：

- `关键词`：匹配公司、城市、岗位、薪资、公司类型、岗位类型。
- `公司`：按公司名称模糊匹配。
- `城市`：按城市模糊匹配。
- `公司类型`：按 `industry` 模糊匹配。
- `岗位类型`：匹配中文输入值和规范化后的内部值。
- `页码`：默认 1。
- `每页`：默认 5，最大 10。

查询结果按 Offer 编号从小到大排序。回复格式示例：

Plain Text

1 第 1/2 页，共 8 条

2 编号：1 | 公司：字节跳动 | 城市：北京 | 岗位：后端开发实习生 | 薪资：300/天 | 学历要求：本科 | 公司类型：互联网 | 岗位类型：实习生

4.5 上传和批量上传设计

单条上传格式：

Plain Text

1 上传 公司= 城市= 岗位= 薪资= 学历要求= 公司类型= 岗位类型=

必填字段：

- 公司
- 城市
- 岗位
- 薪资

选填字段：

- 学历要求
- 公司类型
- 岗位类型

薪资格式要求：

- 300/天
- 25000/月
- 300000/年

批量上传格式：

Plain Text

1 批量上传
2 公司,城市,岗位,薪资,学历要求,公司类型,岗位类型
3 公司,城市,岗位,薪资,-,公司类型,岗位类型

批量上传规则：

- 每行必须正好 7 列。
- 公司、城市、岗位、薪资不能为空，且不能用 - 占位。
- 学历要求、公司类型、岗位类型可以用 - 表示未知。

- 每行独立校验，部分行失败不会影响其他正确行创建。

4.6 岗位类型规范化

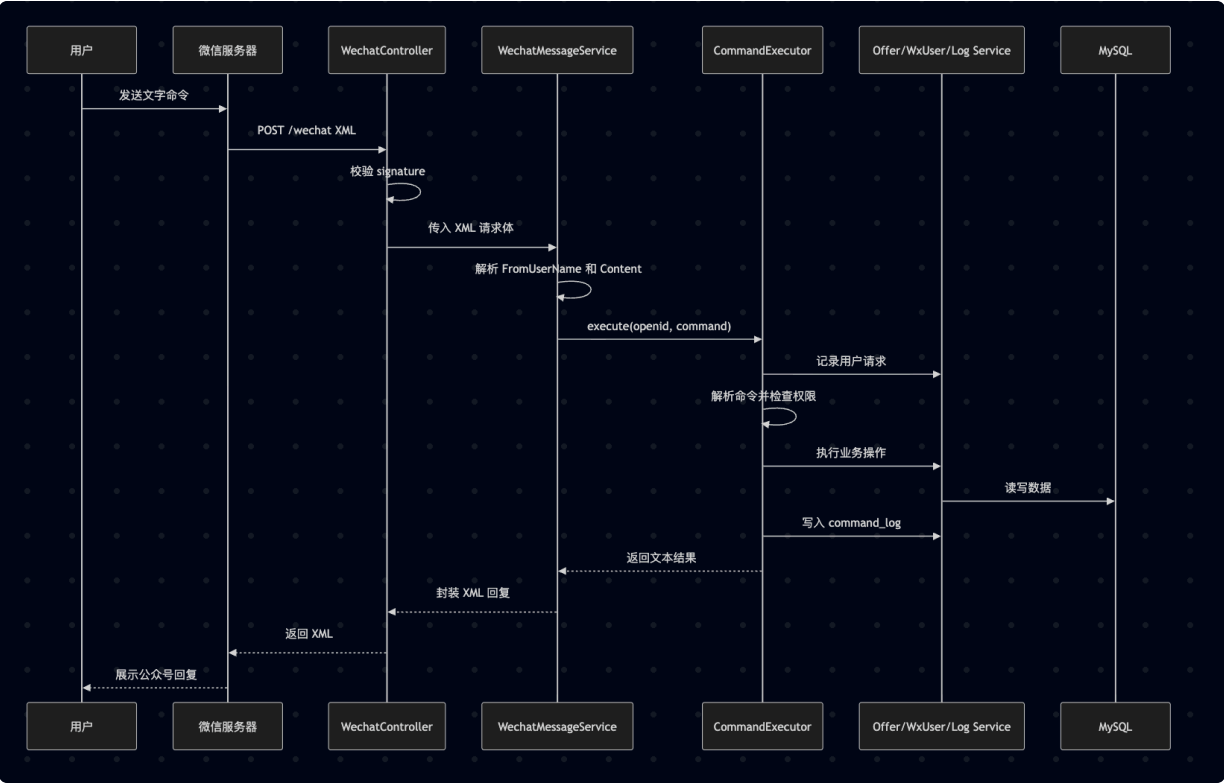
系统内部将常用中文岗位类型转换为规范值：

用户输入	内部存储	展示文本
实习	internship	实习生
校招	campus	校招
社招	fulltime	社招

查询时会同时匹配用户输入值和规范化值。例如 岗位类型=实习 可以匹配数据库中的 internship。

5.核心业务流程设计

5.1 微信消息处理流程



5.2 查询流程

1. 用户发送 `查Offer` 命令。
2. 解析器将字段转换为内部参数。
3. 用户服务记录请求次数，并判断是否触发 `SPIDER`。
4. 执行器检查查询权限。
5. Offer 服务构造查询条件：
 - 关键词使用多字段 `like`。
 - 公司、城市、公司类型使用 `like`。
 - 岗位类型同时匹配原始值和规范化值。
6. 查询结果按编号升序排序，并按页码切片。
7. 执行器格式化为适合微信阅读的短文本。
8. 命令日志记录执行结果。

5.3 上传流程

1. 用户发送 `上传` 命令。
2. 解析器得到 `CREATE` 动作和参数 Map。
3. 执行器检查用户是否被 `BANNED`。
4. Offer 服务校验必填字段。
5. Offer 服务校验薪资格式。
6. Offer 服务将岗位类型等选填字段规范化。
7. 保存 Offer，并将 `source_openid` 设置为当前微信用户的 `FromUserName`。
8. 返回新 Offer 编号。

5.4 更新、替换和删除流程

更新、替换和删除都先按编号读取 Offer，然后进行权限校验。

权限规则：

- 如果当前用户角色为 `ADMIN`，允许操作。
- 如果当前用户不是管理员，必须满足 `actor0openid == offer.source0openid`。
- 如果不满足，返回 `只能修改或删除自己上传的 Offer。`

局部更新只修改用户提供的字段。替换会要求重新提供完整 Offer 信息。删除会直接删除符合权限要求的记录。

5.5 批量上传流程

1. 用户发送多行文本，第一行为 `批量上传`。
2. 解析器将第一行之后的非空行作为批量数据。
3. Offer 服务逐行用逗号拆分。
4. 检查是否正好 7 列。
5. 检查前 4 列必填字段。
6. 将 `-` 规范化为空值。
7. 每一行单独创建 Offer。
8. 返回成功数量和失败数量；失败时返回具体行号和原因。

6. 权限与安全设计

6.1 微信签名校验

系统所有微信 GET 验证请求和 POST 消息请求都需要通过签名校验。校验逻辑如下：

1. 取 Token、timestamp、nonce。
2. 字典序排序。
3. 拼接为字符串。
4. 计算 SHA-1。
5. 与微信传入的 signature 比较。

这可以防止非微信来源直接伪造请求调用接口。

6.2 XML 安全解析

`WechatMessageService` 使用 DOM 解析 XML，并开启安全配置：

- `FEATURE_SECURE_PROCESSING`
- 禁用 DOCTYPE。
- 禁用外部实体。
- 禁用 XInclude。
- 禁用实体展开。

该设计用于降低 XXE 等 XML 解析风险。

6.3 用户身份与权限

微信消息中的 `FromUserName` 被用作用户 `openid`。系统不要求用户主动登录，而是通过微信平台提供的 `openid` 区分用户。

权限分为两层：

- 用户状态权限：限制 `SPIDER` 查询、限制 `BANNED` 写操作。
- Offer 资源权限：普通用户只能修改或删除自己上传的 Offer。

6.4 反爬虫与自动限制机制

系统在 `wx_user` 表中维护用户状态和请求频率窗口。每个微信用户通过 `openid` 唯一识别，后端每次收到命令时都会先调用 `WxUserService.recordRequest(openid)` 更新该用户的请求计数。

当前反爬虫规则为：同一用户在 30 秒内请求次数超过 30 次时，系统会自动将该用户状态从 `NORMAL` 修改为 `SPIDER`，并记录限制原因 `30 秒内请求超过 30 次`。该规则用于识别短时间内高频抓取 Offer 信息的异常访问行为。

被标记为 `SPIDER` 的用户不能继续执行查询类命令，包括：

- 详情
- 列表
- 查Offer
- 找名企

写入类权限由 `BANNED` 状态控制。被标记为 `BANNED` 的用户不能执行写入和修改类命令，包括：

- 上传
- 批量上传
- 更新
- 替换
- 删除

该机制与命令日志配合使用，可以在发现异常访问后保留用户命令轨迹，为后续管理员人工判断、解封或进一步封禁提供依据。对于演示场景，可以通过连续高频发送消息触发 `SPIDER` 状态，也可以由管理员直接在 `wx_user` 表中修改用户状态以展示不同权限效果。

6.5 命令日志

所有命令都会写入 `command_log`。日志包括：

- 输入原文。

- 解析后的动作。
- 成功或失败状态。
- 成功结果或错误原因。

这样可以在演示或调试时快速定位问题，例如命令字段错误、权限失败、格式校验失败等。

7.异常与边界处理

场景	处理方式
空命令或未知命令	返回帮助提示
参数不是 字段=值	返回参数格式错误
未知字段	返回无法识别字段
缺少必填字段	返回缺少字段列表
编号非正整数	返回编号必须大于 0
页码或每页非法	返回分页参数错误
查询无结果	返回无匹配结果和放宽条件提示
薪资格式错误	提示使用 数字/天 、 数字/月 、 数字/年
批量上传列数错误	返回具体行号和 7 列格式要求
普通用户修改他人 Offer	返回权限错误
非文字消息	提示当前仅支持文字消息

业务异常使用 `BusinessException` 表示，并返回明确中文错误提示。未知运行时异常会被统一转换为 `系统暂时无法处理这条命令，请稍后再试。`，避免暴露内部错误。

8.部署设计

8.1 运行环境

组件	版本或说明
JDK	Java 17
后端框架	Spring Boot 3.3.5
数据库	MySQL 8.4
ORM	MyBatis-Plus 3.5.7
数据库迁移	Flyway
构建工具	Maven
容器化	Docker Compose

8.2 配置项

核心配置通过环境变量注入。

环境变量	说明
SERVER_PORT	后端监听端口，默认 8081
WECHAT_TOKEN	微信公众号服务器配置 Token
WECHAT_APP_ID	微信公众号 AppID，当前被动回复功能可为空
WECHAT_APP_SECRET	微信公众号 AppSecret，当前被动回复功能可为空
MYSQL_ROOT_PASSWORD	MySQL root 密码
MYSQL_DATABASE	数据库名称，默认 offer_bot
MYSQL_USER	应用数据库用户，默认 offer_bot
MYSQL_PASSWORD	应用数据库密码
DB_URL	Spring 数据库连接地址
DB_USERNAME	Spring 数据库用户名
DB_PASSWORD	Spring 数据库密码
FLYWAY_ENABLED	是否启用 Flyway 迁移

8.3 Docker Compose 设计

容器编排包含两个服务：

- mysql：使用数据卷 offerbot-mysql-data 持久化数据，字符集设置为 utf8mb4。
- offerbot：依赖 MySQL 健康检查通过后启动，连接 mysql:3306。

数据库端口只绑定到服务器本机 127.0.0.1:3306，避免数据库直接暴露到公网。后端端口也绑定到服务器本机 127.0.0.1:8081，公网访问由 Nginx 反向代理处理。

9.测试设计

系统测试覆盖以下方面：

测试类别	覆盖内容
命令解析测试	中文命令、中文字段、未知字段、非法参数
Offer 服务测试	创建、详情、分页列表、查询、更新、替换、删除
批量上传测试	正确 7 列、少列、多列、必填字段 - 、可选字段 -
权限测试	他人 Offer 不能更新或删除
微信接口测试	GET 验证、POST 文本消息、非文本消息、签名失败
签名测试	SHA-1 排序签名算法
健康检查测试	/health 返回 OK

当前项目使用 H2 内存数据库作为测试数据库，避免测试污染真实 MySQL 数据。

10.设计总结

本系统使用微信公众号作为前端入口，以 Java Spring Boot 实现后端核心逻辑，通过 MySQL 保存 Offer、用户和命令日志。系统将聊天消息抽象为中文命令行，既适合演示，也适合用户快速输入。核心业务围绕 Offer 数据展开，实现了增、删、改、查、替换、批量上传、分页检索、权限控制和错误鲁棒性。